

Contents

SOQL for Admins: sf CLI Workflow	1
Prerequisites	2
What the challenge wants	2
1. Create the Apex class file	2
2. Create the Apex metadata file	3
3. Deploy the class to your org	3
4. Run the method with anonymous Apex	3
5. Optional: create sample Account data if needed	4
6. Useful verification commands	4
7. Fast Neovim workflow	4
8. Minimal command list	5
9. Expected local files	5
What this covers	6
Prerequisites	6
Core Apex concepts	6
Contact example	6
Challenge solution	6
SF CLI workflow	7
1. Create the class file	7
2. Add metadata	7
3. Deploy the class	7
4. Run the method with anonymous Apex	8
5. View logs	8
Neovim workflow with native SF utils	8
Useful Apex utility actions	8
Suggested flow in Neovim	8
Example anonymous Apex scratch buffer	9
Optional SOQL scratch workflow	9
Validation checklist	9
Related examples	9
Contact query in anonymous Apex	9
Contact class example	9
Reference links	10

SOQL for Admins: sf CLI Workflow

This README translates the Trailhead unit **Create SOQL Queries in Apex Classes** into a Neovim + Salesforce CLI workflow. The module is part of **SOQL for Admins**, and the unit asks you to create an Apex class named `AccountUtility` with a public static void method named `viewAnnualRevenue` that queries `Account` records for `Name` and `AnnualRevenue`, loops through the results, stores `Account Name : Annual Revenue` in a variable named `acctRev`, and prints it with `System.debug`.

The Salesforce CLI supports both creating local Apex files and running Apex code, including anonymous Apex and test commands, which makes it a good replacement for the Developer Console workflow described in Trailhead.

Prerequisites

Make sure Salesforce CLI is installed, your org is authenticated, and you are inside a Salesforce DX project directory.

Recommended setup commands:

```
sf org list
sf config set target-org=trailheadPlayground
mkdir -p force-app/main/default/classes
mkdir -p scripts/apex
```

`sf org list` shows your authorized orgs, and `sf config set target-org=...` lets later commands use your default org automatically.

What the challenge wants

The unit's class challenge requires:

- Class name: `AccountUtility`.
- Method name: `viewAnnualRevenue`.
- Method modifiers: `public static void`.
- A list named `accountsList` containing a SOQL query for `Name` and `AnnualRevenue` from `Account`.
- A loop over the query results.^[2]
- A string variable named `acctRev` that stores `Account Name : Annual Revenue`.
- A debug statement that prints `acctRev`.^[2]

1. Create the Apex class file

Open the class in Neovim:

```
nvim force-app/main/default/classes/AccountUtility.cls
```

Paste this Apex code:

```
public with sharing class AccountUtility {
    public static void viewAnnualRevenue() {
        List<Account> accountsList = [
            SELECT Name, AnnualRevenue
            FROM Account
        ];

        for (Account acct : accountsList) {
```

```

        String acctRev = acct.Name + ' : ' + acct.AnnualRevenue;
        System.debug(acctRev);
    }
}

```

This implementation matches the challenge requirements for the class name, method signature, queried fields, loop, `acctRev` variable, and debug output format.

2. Create the Apex metadata file

Salesforce source format stores Apex classes as a `.cls` file plus a matching `-meta.xml` file.

Create the metadata file:

```
nvim force-app/main/default/classes/AccountUtility.cls-meta.xml
```

Paste this content:

```

<?xml version="1.0" encoding="UTF-8"?>
<ApexClass xmlns="http://soap.sforce.com/2006/04/metadata">
  <apiVersion>61.0</apiVersion>
  <status>Active</status>
</ApexClass>

```

3. Deploy the class to your org

Deploy the file with Salesforce CLI:

```
sf project deploy start --source-dir force-app/main/default/classes/AccountUtility.cls
```

The `project deploy start` command deploys local project metadata to your org.

If you prefer deploying the whole classes folder instead, use:

```
sf project deploy start --source-dir force-app/main/default/classes
```

4. Run the method with anonymous Apex

Trailhead usually has you call the class method through an anonymous Apex block when you want to see debug output from a class method.

Create a small runner script:

```
nvim scripts/apex/run_account_utility.apex
```

Paste:

```
AccountUtility.viewAnnualRevenue();
```

Run it:

```
sf apex run --file scripts/apex/run_account_utility.apex
```

The `sf apex run --file` command executes anonymous Apex from a local file, which is the CLI equivalent of opening Execute Anonymous in the Developer Console and clicking Execute.

5. Optional: create sample Account data if needed

If your org has no useful account data yet, you can create a couple of records with Salesforce CLI so the debug output is easier to verify. Salesforce CLI supports data manipulation commands alongside development workflows.

Example:

```
sf data create record --subject Account --values "Name='Acme Tea' AnnualRevenue=500000"
sf data create record --subject Account --values "Name='Herbal House' AnnualRevenue=1250000"
```

Then run the anonymous Apex file again:

```
sf apex run --file scripts/apex/run_account_utility.apex
```

You should see `USER_DEBUG` lines similar to:

```
Acme Tea : 500000
Herbal House : 1250000
```

6. Useful verification commands

To confirm the class is present locally:

```
ls force-app/main/default/classes/AccountUtility.cls
ls force-app/main/default/classes/AccountUtility.cls-meta.xml
```

To stream or inspect logs while you test:

```
sf apex tail log --color
sf apex log list
```

Salesforce CLI includes Apex log commands for viewing and streaming debug logs in the terminal.

To query Accounts directly from the CLI:

```
sf data query --query "SELECT Name, AnnualRevenue FROM Account" --result-format table
```

Salesforce CLI supports SOQL-based data queries, which is useful for checking the exact records your Apex method should iterate over.

7. Fast Neovim workflow

1. Edit `force-app/main/default/classes/AccountUtility.cls`.

2. Save the file.
3. Run your deploy mapping or execute `sf project deploy start --source-dir force-app/main/default/classes/AccountUtility.cls`.
4. Edit `scripts/apex/run_account_utility.apex` if needed.
5. Run `sf apex run --file scripts/apex/run_account_utility.apex`.
6. Read the `USER_DEBUG` output in the terminal.

If you already added Neovim commands like `:SfDeployFile` and `:SfApexRun`, the loop becomes:

```
:edit force-app/main/default/classes/AccountUtility.cls
:write
:SfDeployFile
:edit scripts/apex/run_account_utility.apex
:write
:SfApexRun
```

8. Minimal command list

If you only want the exact commands to complete the module workflow, use this sequence:

```
sf org list
sf config set target-org=trailheadPlayground
mkdir -p force-app/main/default/classes scripts/apex
nvim force-app/main/default/classes/AccountUtility.cls
nvim force-app/main/default/classes/AccountUtility.cls-meta.xml
sf project deploy start --source-dir force-app/main/default/classes/AccountUtility.cls
nvim scripts/apex/run_account_utility.apex
sf apex run --file scripts/apex/run_account_utility.apex
```

9. Expected local files

Your project should end up with at least these files:

```
force-app/main/default/classes/AccountUtility.cls
force-app/main/default/classes/AccountUtility.cls-meta.xml
scripts/apex/run_account_utility.apex
```

That structure matches Salesforce DX source format for an Apex class plus an anonymous Apex runner script.

Salesforce SOQL in Apex: Contact and Account Utility Walkthrough

This guide turns the Trailhead-style SOQL-in-Apex exercise into a practical workflow using the Salesforce CLI and a Neovim setup with native Salesforce utility commands. SOQL queries in Apex return lists of sObjects, and those lists can be iterated with `for` loops to format and debug the result set.[page:1]

What this covers

The Salesforce learning flow shown here builds from a simple SOQL query into an Apex class method that stores query results in a list, loops over the records, and prints formatted output with `System.debug`. The final hands-on challenge asks for an `AccountUtility` class with a `viewAnnualRevenue` method that queries `Name` and `AnnualRevenue` from `Account`, stores the results in `accountsList`, and logs a formatted `acctRev` string for each record.

Prerequisites

- Salesforce CLI (`sf`) installed and authenticated to a target org.
- A Salesforce DX project containing `sfdx-project.json`, which is the workspace marker used by the Neovim utility layer.
- Neovim configured with the native Salesforce utility modules such as `utils.sf.util`, `utils.sf.apex`, `utils.sf.tests`, and optional SOQL/SOSL helpers.

Core Apex concepts

A SOQL query embedded in Apex is wrapped in square brackets and assigned directly to a typed list such as `List<Contact>` or `List<Account>`. Once the results are in a list, a `for` loop can iterate record-by-record and use dot notation like `con.FirstName` or `acct.AnnualRevenue` to construct debug output strings.

Contact example

The Contact example in the lesson stores query results in `listOfContacts`, then loops through each `Contact` to build a `fullName` string and print it with `System.debug`.

```
public class ContactUtility {
    public static void viewContacts() {
        List<Contact> listOfContacts = [SELECT FirstName, LastName FROM Contact];
        for (Contact con : listOfContacts) {
            String fullName = 'First Name: ' + con.FirstName + ', Last Name: ' + con.LastName;
            System.debug(fullName);
        }
    }
}
```

Challenge solution

The challenge requires an `AccountUtility` class and a `viewAnnualRevenue` method that queries `Name` and `AnnualRevenue` from `Account`, stores the records in `accountsList`, then prints `Name : AnnualRevenue` for each account.

```

public class AccountUtility {
    public static void viewAnnualRevenue() {
        List<Account> accountsList = [SELECT Name, AnnualRevenue FROM Account];
        for (Account acct : accountsList) {
            String acctRev = acct.Name + ' : ' + acct.AnnualRevenue;
            System.debug(acctRev);
        }
    }
}

```

SF CLI workflow

In a local Salesforce project, create the Apex class file under `force-app/main/default/classes/AccountUtility.cls` and add the class body shown above. Then create the matching metadata file `AccountUtility.cls-meta.xml` if your project structure does not already generate it.

1. Create the class file

```

mkdir -p force-app/main/default/classes
nvim force-app/main/default/classes/AccountUtility.cls

```

Paste in:

```

public class AccountUtility {
    public static void viewAnnualRevenue() {
        List<Account> accountsList = [SELECT Name, AnnualRevenue FROM Account];
        for (Account acct : accountsList) {
            String acctRev = acct.Name + ' : ' + acct.AnnualRevenue;
            System.debug(acctRev);
        }
    }
}

```

2. Add metadata

```

<?xml version="1.0" encoding="UTF-8"?>
<ApexClass xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>63.0</apiVersion>
    <status>Active</status>
</ApexClass>

```

Save that as:

```

force-app/main/default/classes/AccountUtility.cls-meta.xml

```

3. Deploy the class

```

sf project deploy start --source-dir force-app/main/default/classes/AccountUtility.cls

```

If you want to deploy the whole classes directory instead:

```
sf project deploy start --source-dir force-app/main/default/classes
```

4. Run the method with anonymous Apex

Create a scratch file such as `scripts/run_account_utility.apex`:

```
AccountUtility.viewAnnualRevenue();
```

Run it with:

```
sf apex run --file scripts/run_account_utility.apex
```

5. View logs

List recent Apex logs:

```
sf apex log list
```

Get a specific log:

```
sf apex log get --log-id <LOG_ID>
```

Tail logs live while testing:

```
sf apex tail log --color
```

Neovim workflow with native SF utils

If the native utility layer is loaded with `require('utils.sf').setup()` or equivalent command registration, the same flow can be done without leaving Neovim. The utility layer checks for the `sf` executable and supports running anonymous Apex, deploying files, and viewing logs directly from terminal splits.

Useful Apex utility actions

- Run current anonymous Apex script buffer: use the native Apex runner for `.apex` scratch files.
- Run a visual selection as anonymous Apex: useful for quick one-off tests.
- Tail or fetch logs from inside Neovim after execution.
- Deploy the current class file from the current buffer when editing `AccountUtility.cls`.

Suggested flow in Neovim

1. Create `force-app/main/default/classes/AccountUtility.cls`.
2. Write the class body.
3. Save and deploy with your Salesforce file deploy command.
4. Open a temporary `.apex` buffer containing `AccountUtility.viewAnnualRevenue();`.
5. Run the buffer through the native Apex utility.

6. Open the resulting log output or tail the live log stream.

Example anonymous Apex scratch buffer

```
AccountUtility.viewAnnualRevenue();
```

If your `utils.sf.apex` module is wired to commands like `:SfApexRun`, place that line in a `.apex` buffer and run it from the current file. If your config also supports visual execution, select the line and run the selection command instead.

Optional SOQL scratch workflow

For simple query testing before writing Apex, create a `.soql` buffer with a query such as the following and run it through your SOQL command layer:

```
SELECT Name, AnnualRevenue
FROM Account
LIMIT 25
```

SOQL is appropriate when the object and fields are known, and it supports filtering, sorting, counting, and selecting fields from records.[page:1] Adding LIMIT is a good habit for ad-hoc scratch queries, especially in editor-driven workflows.

Validation checklist

- `AccountUtility.cls` exists in the classes directory.
- The class is deployed successfully.
- `AccountUtility.viewAnnualRevenue();` runs without compile errors.
- The debug log shows lines formatted as `Account Name : AnnualRevenue`.
- Optional: the query works first in a `.soql` scratch buffer before being moved into Apex.

Related examples

Contact query in anonymous Apex

```
List<Contact> listOfContacts = [SELECT FirstName, LastName FROM Contact LIMIT 2];
System.debug(listOfContacts);
```

If no records are found, the result is still a list, but it is empty.[page:1]

Contact class example

```
public class ContactUtility {
    public static void viewContacts() {
        List<Contact> listOfContacts = [SELECT FirstName, LastName FROM Contact];
        for (Contact con : listOfContacts) {
```

```
        String fullName = 'First Name: ' + con.FirstName + ', Last Name: ' + con.LastName;
        System.debug(fullName);
    }
}
```

Reference links

- [Salesforce SOQL and SOSL introduction](#)
- [Apex Developer Guide: Classes](#)
- [Apex Developer Guide: Class Methods](#)